

P3565R1: Virtual floating-point values

Audience: SG6

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 15, 2025

History

Since [r0](#):

- Added FMA example with + explanation
- Discussed relationship with FLT_EVAL_METHOD
- Discussed effect on CWG2752
- Discussed implementation requirements

Introduction

The standard says very little about the actual results of floating-point evaluations. [basic.fundamental]/12 says the “accuracy of operations” is implementation-defined; [expr.pre]/6 makes the situation even less clear by suggesting that floating-point operands and results are somehow not even values of their types. Indeed, it is of practical value that implementations often interpret expressions involving floating-point types as mathematical expressions in order to improve the performance and accuracy of the computation of its overall result. Common techniques include fusing multiplications and additions, discarding canceling terms, and temporarily using extra precision (*e.g.*, using x87 registers). Strict application of well-defined floating-point operations is of course critical to other numerical algorithms; the footnote in /6 suggests that “The cast and assignment operators” may be used for the purpose.

These ideas are derived from C, which additionally defines the FLT_EVAL_METHOD macro to describe the implementation’s choices about such transformations. Matthias Kretz presented [Floating-Point Excess Precision](#) to SG6 and EWG seeking guidance on how to most consistently interpret these ideas (in particular about floating-point literals, the subject of [CWG2752](#)) in the context of C++’s stronger type system, constant evaluation, and the larger set of contemporary floating-point types. No clear direction has yet been reached, suggesting that further research may be needed.

This paper proposes a model for the use of extra precision (when available) for floating-point calculations that is more appropriate for C++ than `FLT_EVAL_METHOD` and resolves CWG2752. It does not, however, suggest mandating any particular arithmetic specification like IEC 60559.

Discussion

The idea that an operator result of a type does not have one of the values of that type is obviously problematic from the perspective of defining semantics for such an operator. Moreover, the idea that assigning a variable forces extended precision to be discarded is problematic in C++ because of the routine use of wrapper class types in mathematical expressions. The creation of every such object involves the initialization of its member variable, which seems to be just as strong as assignment in terms of incompatibility with extralinguistic extended precision.

An alternative approach is to extend the set of values for a floating-point type beyond those that can even theoretically be stored in the memory that an object of that type occupies. The result of the subexpression in `a * b + c` (all doubles) might then have a value outside the set of values that can be stored in a double's space in memory (typically the `binary64` set); the choice of that value conveys the additional information needed to obtain the correctly rounded result of the overall expression as of course implemented by an FMA instruction. Similar careful choices of value from a larger set might capture the bits (or finiteness) lost in `a + b - b`; for x87 implementations, the larger set is simply the values supported by the register format. `FLT_EVAL_METHOD==1` corresponds to using the normal set of doubles as the extended set for `float`.

The crucial specification technology is the same as used for [pointer provenance](#): the values are put in a many-to-one correspondence with the value representations of the type. (The presence of multiple rounding modes might require the formal duplication of values based on the representable value to which they round, but this matters only if the value representation is examined.) Note that every operation and object still has a *single* value. Aside from merely being tractable semantics, the stability of values prevents unfortunate practical results like taking both branches in

```
const double x = /* ... */, y = x + epsilon / 4;
if(x < y) {      // X87 comparison
    // ...
}
// further operations that cause spilling...
if(x == y) {     // binary64 comparison
    // ...
}
```

or failing the assert in

```
float id(float f) {return f;} // no computations
void call() {
    const float x = /* ... */;
    assert(3 * x == 3 * id(x));
}
```

Note the implication that if `id` is not inlined `x` must be given a representable value for consistency. When accuracy is more important than consistency, the unary `+` may be used to perform a trivial calculation to be (re)rounded:

```
void sink(double); // not inlined
double tee(double x, double y, double a) {
    const double m = x * y;
    sink(+m);
    return m + a;
}
```

Because `sink` need not be provided the exact value of `m`, an FMA may still be used for the return value. An unusually aggressive implementation might exploit the unspecified accuracy of even the `+` operation to perform the same optimization if the `+` appeared instead in the return statement, but that would not be consistent with advertising arithmetic conforming to (say) IEC 60559.

For obvious practical reasons, a value that escapes past an optimization frontier cannot actually store information beyond its bit pattern. The stability requirement implies that any such value must be normalized to its “memory type” upon computation. However, even the member variables of wrapper objects can have an extended value if the optimizer sees their entire lifetime (as is typical for temporaries in the sorts of expressions we want to optimize freely), because they truly are members of the type. Similarly, assignment does not need the normalization effect described in the [expr.pre]/6 footnote; even a value updated in a loop may be extended so long as its intermediate values do not escape. Values passed to and returned from standard-library mathematical functions can also be extended.

Even literals are subject to the same rules: those that appear as operands or that are used only locally can preserve extra digits with any available extra precision, while those that “escape” must be represented in the ordinary fashion for their type for consistency. One means of such escape is a constant variable that might be used in multiple translation units (because it is inline, appears in an importable module unit, or is used in a template argument), unless whole-program optimization can arrange for some particular, more accurate value to be used in all cases.

As there are no opaque functions (merely insufficiently clever optimizers), it is only prudent to retain an explicit means of requiring normalization; `static_cast` is the obvious candidate (the other part of the footnote), although `std::memcpy` would also have the effect of selecting the canonical value associated with a value representation. (For pointers, `std::memcpy` needs to be able to preserve the abstract-machine information to prevent undefined behavior, but here it would be unnecessary and difficult to specify since it does affect observable behavior.)

Implementation issues

While it would be a conforming implementation of the existing standard and of this proposal to spill all values immediately and to never use fused multiply-adds, generating performant code with this proposal requires the implementation to track the usage of existing values rather than to merely round, and avoid FMA contraction, when encountering casts and assignments. There is an analogy to escape analysis of pointers: when a use of a value requires that it be representable in the in-memory format (perhaps because it crosses beyond the current optimization frontier), the implementation must

arrange for that value to be appropriately rounded when it is computed or for it to be excluded from FMA contraction. (In a sufficiently long function, it might be difficult to maintain that information about all values; it is permissible to conservatively apply the treatment for values that escape to any that have not been proven not to escape.)

In the SG6 discussion of this paper in Hagenberg, it was pointed out that some existing implementations do not maintain in their intermediate representations adequate information to identify when a value is spilled from an X87 register (and thus rounded to its memory representation) or involved in an FMA contraction. However, it should be noted that existing implementations do not conform to the current wording (which is both inadequate and questionably normative) by default, even with options such as `-std=c++...-pedantic`; in that regard, this proposal can be said to improve the alignment with existing implementations (in the absence of special options like `-ffp-contract=on` (which despite its name actually reduces the use of FMA)) in that assignment (or initialization) is no longer considered to necessitate the loss of any excess precision.

Proposal

Modify the definition of trivially-copyable types to allow floating-point types to have multiple values (one of which is canonical) per value representation. Specify that acquiring a value representation gives a floating-point object the corresponding canonical value.

Replace the “greater precision and range” provision ([`expr.pre`]/6) with a note about the contextual dependence of rounding. Specify that unary `+` can, and that `static_cast` does, round floating-point values to their corresponding canonical values.